WEB SECURITY SCANNER

Automated Web Security Configuration Assessment Using Python

ACADEMIC PROJECT REPORT

Student Name	Abhijeet Sanjay Khot
Institution	MITWPU

Project summary

A Python command-line tool that performs selected web security configuration checks, reports preliminary findings, and exports results to JSON and HTML. The implementation uses the Requests library for HTTP requests and pytest for automated tests.

This report documents the implementation and results supplied by the student; it does not claim a full penetration test or independently verified vulnerability assessment.

Abstract

This project implements a lightweight Python-based Web Security Scanner for checking selected HTTP security configuration signals. The scanner accepts an HTTP or HTTPS URL, validates its basic structure, sends a request with a timeout and TLS certificate verification enabled, and inspects the final response for HTTPS usage, selected security headers, and a basic wildcard CORS condition. Findings include a severity label, evidence, and a recommendation. The tool exports a structured JSON report and a readable HTML report. The submitted test run collected 14 automated tests, all of which passed. A scan of the supplied GitHub Pages URL returned HTTP status 200 and produced five preliminary findings. These findings are configuration observations, not proof that the target is exploitable.

1. Introduction

Web applications rely on secure transport and appropriate HTTP response configuration to reduce risks such as content injection, unsafe browser behavior, and unintended cross-origin access. Manual inspection can be repetitive, so a small automated tool can help identify selected configuration issues and produce consistent evidence for review.

The project focuses on a narrow, understandable set of checks rather than attempting to replace professional web application testing. Its goal is to provide an educational demonstration of Python automation, basic web security concepts, test-driven validation, and report generation.

2. Objectives

• Validate that an input is a complete HTTP or HTTPS URL with a hostname.

• Request the target with a timeout and TLS certificate verification enabled.

• Inspect HTTPS redirect behavior and selected response security headers.

• Flag a wildcard Access-Control-Allow-Origin response for contextual review.

• Generate JSON and HTML reports containing findings and remediation guidance.

• Use automated tests to check URL validation, scanner behavior, and report generation.

3. Tools and Technologies

Technology	Role in the project
Python 3.14.6	Main implementation language used in the recorded Windows environment.
Requests 2.34.2	Sends HTTP requests and exposes response status, URL, and headers.
pytest 9.1.1	Runs automated tests for scanner and report behavior.
HTML / CSS	Presents scan results in a browser-readable report.
JSON	Stores structured scan results for programmatic inspection.
Visual Studio Code on Windows	Development environment used for implementation and execution.

4. System Design and Workflow

The implementation is divided into three principal modules. scanner.py validates input, sends the request, performs configured checks, and returns a structured result. rules.py defines the security headers to inspect and provides descriptions and recommendations. report.py sorts findings by configured severity and writes JSON and HTML output files.

Input URL
Basic URL validation
HTTP request (10-second timeout; TLS verification enabled)
Inspect final URL, status, configured headers, and basic CORS response
Return structured findings with evidence and recommendations
Write sample_reports/report.json and sample_reports/report.html

4.1 Implemented checks

• HTTPS: flags a submitted HTTP URL and checks whether an HTTPS request ends at a non-HTTPS final URL.

• Security headers: checks for Strict-Transport-Security, Content-Security-Policy, X-Content-Type-Options, Referrer-Policy, and Permissions-Policy.

• Header value: checks whether X-Content-Type-Options is set to nosniff when the header is present.

• CORS: detects Access-Control-Allow-Origin: * and reports its presence as a preliminary observation. It does not test exploitability.

4.2 Output files

report.json stores target URL, final URL, HTTP status, HTTPS status, findings, and scan timestamp. **report.html** presents a summary and finding cards with evidence and recommendations. The HTML report escapes untrusted text before inserting it into the document.

5. Implementation Details

The scanner uses urllib.parse to perform basic URL validation and the Requests library to retrieve the response. Requests is configured with a 10-second timeout, redirects enabled, TLS verification enabled, and a descriptive User-Agent. Network-related Requests exceptions are converted into a clear scan error. Results are returned as a Python dictionary, which is then passed to the report generator.

Severity labels are assigned by the current project rules. They are intended to prioritize review and should not be interpreted as a formal CVSS score or as a statement that a vulnerability is confirmed.

6. Testing Methodology and Results

Automated tests were executed in the project's Windows virtual environment using pytest. The recorded test command was `python -m pytest -v`. The terminal output reported **14 passed in 0.28s**.

Test group	Coverage	Result
URL validation	Accepts HTTP/HTTPS URLs and rejects malformed, empty, or unsupported URLs.	Passed
Missing header detection	Checks that absent configured security headers create findings.	Passed
Wildcard CORS detection	Checks that a wildcard origin produces a CORS finding.	Passed
Scanner invalid URL handling	Checks that scan_website rejects invalid input.	Passed
Report file creation	Checks that JSON and HTML files are written.	Passed
JSON report content	Checks scan data, finding severity, and timestamp in JSON.	Passed
HTML report content	Checks finding details, recommendations, and severity summary.	Passed
HTML escaping	Checks that script-like text is escaped in generated HTML.	Passed
Empty findings report	Checks that a report with no findings is generated.	Passed

Recorded automated test result: 14 tests passed; no failures were shown in the supplied pytest output.

7. Demonstration Scan Results

The student ran the scanner against the supplied GitHub Pages URL `https://abhijeet01khot.github.io/FSD-Assignment-4/`. The recorded output showed HTTP status 200, the same final URL, HTTPS used = True, and five findings.

Finding reported	Severity shown by tool	Interpretation
Content-Security-Policy missing	Medium	The response lacked this header; suitability depends on the application and its content sources.
X-Content-Type-Options missing	Low	The response lacked the header that can help prevent MIME sniffing.
Referrer-Policy missing	Low	The response did not explicitly provide this header.
Permissions-Policy missing	Informational	The response did not explicitly provide this browser-feature policy header.
Wildcard CORS origin detected	Informational	Access-Control-Allow-Origin was '*'; credentials header was not set. This can be intentional for public resources.

Interpretation: These are five findings reported by the tool, not five confirmed vulnerabilities. The scan was a single response inspection and did not establish exploitability. The CORS observation in particular requires context; wildcard origin is commonly used for public static content.

8. Limitations and Ethical Considerations

• The scanner checks only a selected set of headers and simple response conditions.

• It does not crawl pages, inspect client-side JavaScript, test authentication or authorization, or assess business logic.

• It does not validate the effectiveness of a Content-Security-Policy or fully assess CORS behavior.

• The reported severity labels are simple project-defined labels and not a formal risk-scoring methodology.

• A response can vary by route, user agent, CDN, request method, or application state; one scan may not represent the entire site.

• Only systems owned by the student or explicitly authorized for testing should be scanned.

9. Future Improvements

• Validate more header values and provide contextual checks for HSTS and CSP.

• Add explicit unit tests for HTTP errors, redirects, and request timeouts.

• Improve CORS analysis with controlled preflight and origin-specific tests in an authorized lab.

• Add command-line arguments for the target URL, timeout, and report output directory.

• Add a local intentionally vulnerable test application to provide repeatable positive and negative test cases.

• Add continuous integration to run pytest on each repository update.

10. Conclusion

The project demonstrates a functioning Python workflow for selected web security configuration checks and report generation. The scanner returned HTTP status 200 and five preliminary findings for the demonstration URL. Its JSON and HTML reporting functions were exercised, and the supplied automated test run reported 14 passing tests. The result is a useful educational foundation for learning HTTP headers, basic CORS review, test automation, and structured security reporting. It should be treated as a preliminary configuration checker rather than a replacement for a full security assessment.

Appendix A: Commands Used

Create a virtual environment and install dependencies:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install requests pytest
```

Run the scanner and automated tests:

```
python scanner.py
python -m pytest -v
```

Appendix B: Evidence to Attach

• Screenshot of the terminal showing the completed demonstration scan.

• Screenshot of pytest showing 14 passed.

• Screenshot of sample_reports/report.html open in a browser.

• Screenshot of sample_reports/report.json open in VS Code.

The screenshots are not embedded in this report because they were not provided as files. Insert your genuine screenshots before submission if visual evidence is required.